

内存虚拟化、地址翻译与页生命周期

从连续重定位、分页、TLB、缺页异常到 COW、Swap、mmap 与物理页分配

408 专题手册 · 操作系统 × 计算机组成原理桥梁篇

核心模型：虚拟内存是一套地址意义管理系统

程序只产生虚拟地址；操作系统定义这些地址代表什么，
硬件高速执行翻译；异常路径再把控制权交回内核。

分析主线：

Virtual Region → Mapping → Physical Residency
→ Translation → Access / Fault

其中必须始终区分四件事：

Allocation ≠ Mapping ≠ Residency ≠ Translation

一个虚拟地址范围可以已经“属于进程”，却尚未有物理页；一个页面可以曾经驻留、后来被换出；一个映射可以合法存在，但当前访问权限仍不允许本次操作。

目录

1 第一原则：先把五种“内存问题”拆开

1.1 先消除“分配”一词的层次混淆

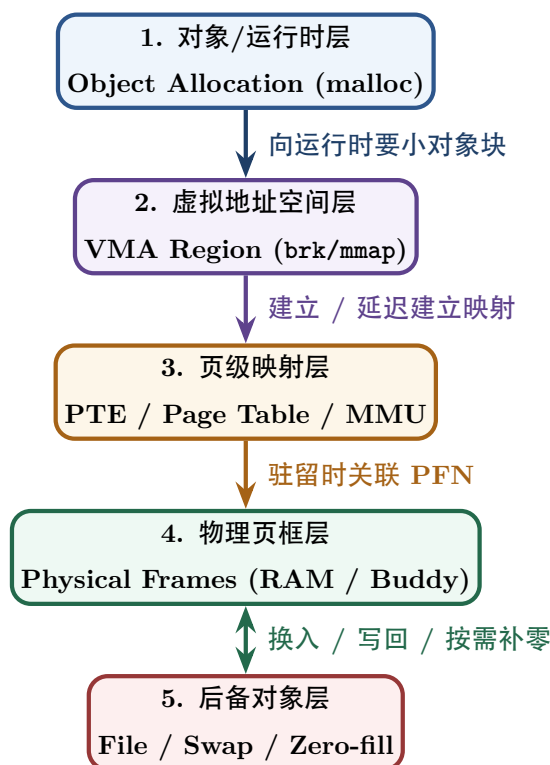
同一个“分配”一词，至少可能指五件完全不同的事：

内存概念层次	管理的对象与单位	核心解决的核心问题	典型代表机制 / 数据结构
语言/运行时对象分配	Object / Chunk	malloc(100) 从哪里得到一块可用小块？	Heap Allocator、Free List、Size Class
进程虚拟地址空间	VA Range / VMA	哪些虚拟地址属于本进程？权限与属性是什么？	VMA (vm_area_struct)、brk、mmap
页级地址映射	VPN → PFN	某个虚拟页号 (VPN) 当前映射到哪个物理页框？	多级页表 (Page Table) / 页表项 (PTE)
物理内存框分配	Physical Frame (PFN)	内核如何从全局空闲物理页中分配连续页？	Free Page List、Buddy System 伙伴系统
驻留与页面回收	Resident Page	主存紧张时，谁保留在 RAM、谁写回/淘汰？	Page Reclaim、Page Replacement、Swap/Writeback

最容易形成的错误模型

malloc() 并不等于“每次调用系统调用，请 OS 用 First Fit 找一块物理内存”。现代用户态分配器通常先管理已经取得的虚拟地址区域；只有需要扩展 arena 或建立新映射时，才通过 brk/mmap 等接口向内核请求地址空间。即使地址空间已经扩大，也可能通过按需分配直到首次访问才真正获得物理页。

1.2 本册统一五层模型



每层回答不同问题

- VMA: 这个 VA 是否属于进程? 允许 R/W/X 吗? 背后是什么对象?
- PTE: 这个虚拟页当前怎样翻译? 权限和驻留状态是什么?
- Physical Frame: 真实 RAM 中哪一页现在承载数据?
- Backing Object: 如果页不在 RAM, 数据从哪里恢复? 文件、Swap, 还是直接补零?

2 为什么必须虚拟化地址: 四个矛盾

2.1 重定位: 程序不应该知道自己被放在 RAM 的哪里

最早的核心问题可以写成:

$$\text{Program Address} \neq \text{Physical Placement.}$$

若程序内部写死物理地址, 那么每次换装入位置都必须修改程序; 运行中移动、交换、共享更困难。

最简单的动态重定位模型是:

$$PA = Base + VA, \quad 0 \leq VA < Limit.$$

硬件完成加法与越界检查, OS 在切换进程时配置 Base/Limit。

2.2 隔离: 不同进程可以使用“相同地址”, 却不应访问同一内存

例如两个进程都可以访问自己的:

$$VA = 0x400000.$$

通过不同映射：

$$VPN_A \rightarrow PFN_{12}, \quad VPN_B \rightarrow PFN_{91},$$

它们的地址数值相同，物理对象却完全不同。

2.3 共享：隔离不是“不许共享”，而是共享必须显式受控

若只读共享库页需要被多个进程使用，可以让：

$$VPN_A \rightarrow PFN_{37}, \quad VPN_B \rightarrow PFN_{37}$$

并设置只读/可执行权限。于是：

独立虚拟地址空间 $\nRightarrow$ 所有物理页都独占

2.4 稀疏与按需：地址空间可以很大，但真正付费的只是一部分

虚拟地址空间允许大量“洞”和尚未驻留的区域。程序可以先取得一个大虚拟范围，仅在实际访问时再分配物理页：

Reserve VA $\rightarrow$ First Touch $\rightarrow$ Page Fault $\rightarrow$ Allocate Frame.

虚拟内存的三个最重要目标

从系统设计角度，虚拟内存同时追求：

1. **Transparency**：程序以为自己拥有简单、连续的地址空间；
2. **Protection**：不同主体不能越权访问；
3. **Efficiency**：常见地址访问必须靠硬件快路径完成，而不能每次进入内核。

3 从连续重定位到分页：为什么方案一步步升级

3.1 静态重定位：在装入时一次性改地址

经典静态重定位在 load time 修改需要重定位的位置，使程序以后直接使用最终地址。优点是执行期硬件简单；缺点是运行后移动困难，灵活性和隔离能力有限。

3.2 动态重定位：把“地址意义”推迟到运行时

动态重定位把逻辑地址保留下来，直到每次访问时由 MMU 翻译。最简单实现是 Base/Limit。

408 辨析：静态/动态“内存分配”与静态/动态“重定位”不是一对概念

语言层面的静态变量、栈变量、heap 动态分配，讨论的是对象何时取得存储空间；静态/动态重定位讨论的是程序地址在什么时候变成物理地址。两组术语不能混成同一维度。

3.3 连续分配的问题：物理连续性成为束缚

如果每个进程必须占用一段连续物理区，就会出现外部碎片：

$$\sum Free_i \geq Request \quad \text{但} \quad \max Free_i < Request.$$

这促使系统放弃“进程在物理内存必须连续”的要求。

3.4 分页的关键跨越：固定粒度 + 非连续物理放置

分页把：

- 虚拟地址空间切成 page；
- 物理内存切成同样大小的 frame；
- 通过 page table 建立任意 VPN → PFN 映射。

因此进程物理页可以散落在 RAM 中，而虚拟空间依旧看起来连续。

4 分页数学模型：地址为什么要拆成 VPN 与 Offset

4.1 页内偏移保持不变

若页大小为：

$$P = 2^p \text{ Bytes},$$

则虚拟地址可以写成：

$$\boxed{VA = VPN \mid Offset}, \quad |Offset| = p.$$

页表只翻译页号：

$$VPN \rightarrow PFN,$$

而页内 offset 不变：

$$\boxed{PA = PFN \mid Offset}.$$

地址题第一反应

1. 先由 page size 得 offset 位数；
2. 再用虚拟地址总位数减去 offset 得 VPN 位数；
3. 若多级页表，再按每级索引位数继续拆 VPN；
4. 最后不要漏掉 offset 原样拼回物理地址。

4.2 单级页表为什么会太大

若虚拟地址 v 位、页大小 2^p ，虚拟页数：

$$N_{page} = 2^{v-p}.$$

若每个 PTE 占 e Bytes，则单级页表大小：

$$S_{PT} = 2^{v-p} \cdot e.$$

即使实际只用很少地址，也必须为整个 VPN 空间准备表项，因此出现多级页表。

5 多级页表：用稀疏数据结构描述稀疏地址空间

5.1 核心思想不是“多查几次”，而是“未使用的区域不建低级表”

一个多级地址可以抽象成：

$$VA = I_1 | I_2 | \cdots | I_k | Offset.$$

每一级索引选择下一张表，只有实际使用的虚拟区域才需要逐层建立低级页表页。

多级页表本质：树形间接层

多级页表拿更多查询步骤换更少的页表空间。它与文件系统的 inode 间接块、目录树、B+ 树一样，都是“稀疏大命名空间”的层次化描述方法。

5.2 PTE 不只是 PFN

应把 PTE 看成：

$$PTE = Mapping + Permission + State.$$

典型信息包括：

- 物理页框号或下一层页表地址；
- present/valid；
- user/supervisor；
- read/write/execute；
- accessed/reference；
- dirty；
- 架构或 OS 自定义状态。

一条访存真正提出的四个问题

1. 这个虚拟地址属于当前地址空间吗？
2. 本次访问类型（R/W/X）被允许吗？
3. 对应数据当前是否 resident？
4. 如果 resident，它映射到哪个 physical frame？

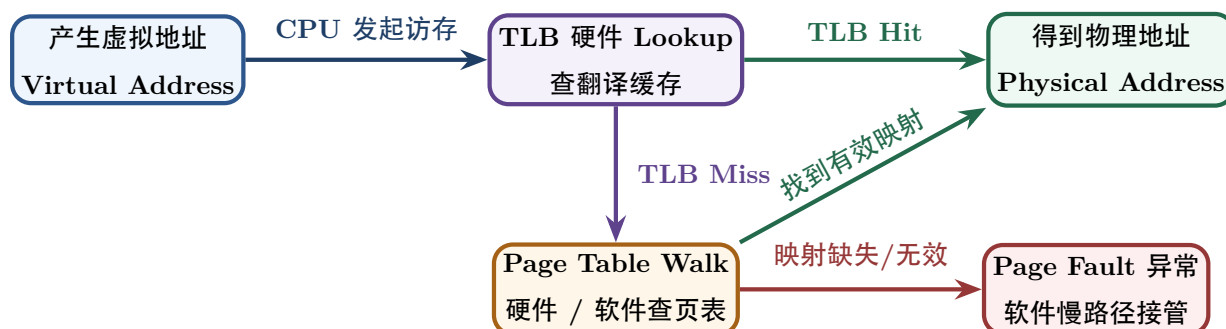
6 TLB：翻译本身也需要缓存

6.1 为什么页表不能每次都完整查

如果每次 load/store 为了得到 PA 都先访问多级页表，那么一次数据访问会附带多次额外内存访问。TLB 用一个小型硬件 cache 保存近期地址翻译：

$$VPN \rightarrow PFN/permission.$$

6.2 快路径与慢路径



6.3 三种 miss/fault 必须彻底分开

现象	缺失的核心内容	是否必然进入 OS 内核?
TLB Miss	硬件地址翻译缓存项 ($VPN \rightarrow PFN$)	不一定；硬件 Page Walk 成功时直接补全 TLB，不触发 OS 异常
Page Fault	当前访问无法沿正常页表路径完成	必须进入 ；需要 OS 捕获异常、分配页框/解装入/COW 或终止
Cache Miss	目标数据 Cache line 不在 L1/L2/L3 Cache	不进入 ；属于存储层次硬件行为，由总线与主存完成加载

典型错误

TLB Miss 不等于 Page Fault；Page Fault 也不等于“页面一定在磁盘上”。COW、lazy allocation、权限错误都可以触发 page fault，而数据可能从未存在于磁盘。

6.4 有效访问时间 EAT 的模型意识

408 常用理想化模型：

$$EAT = h \cdot T_{hit} + (1 - h) \cdot T_{miss}.$$

关键不是背固定式子，而是明确每条路径到底发生几次 TLB 查询、页表内存访问和最终数据访问。题目改变“TLB 与 Cache 是否并行、页表几级、TLB miss 是否硬件 walk”等假设时，公式必须重新从事件路径推导。

工程边界：ASID / PCID

地址空间切换后必须确保旧 TLB 项不会被错误用于新地址空间，但这并不意味着现代 CPU 每次 context switch 都必须“完全清空 TLB”。ASID/PCID 一类标签允许不同地址空间的翻译共存，从而降低切换成本。408 先掌握“一致性必须维护”，再把标签机制作为现代扩展。

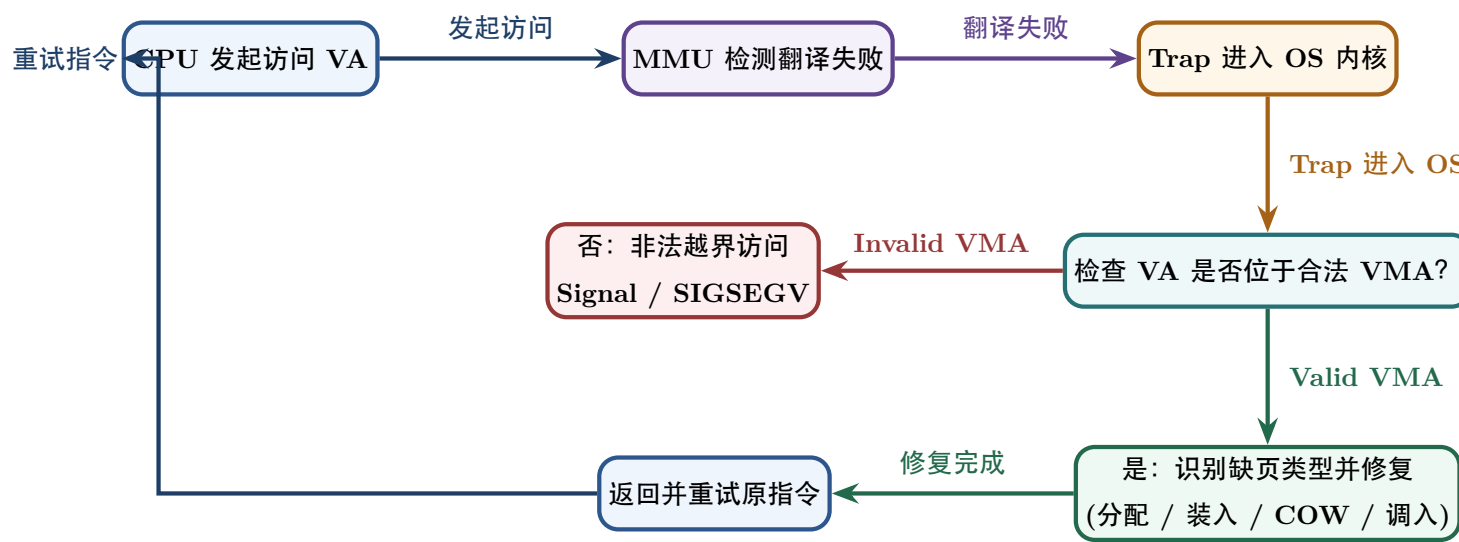
7 Page Fault：虚拟内存的慢路径入口

7.1 定义：不是错误，而是“硬件无法完成当前翻译/访问”

MMU 在正常快路径不能完成访问时触发 page-fault exception。内核收到后必须先分类：

合法但需补工作 or 非法访问

7.2 统一处理流程



7.3 Page Fault 六类母模式

缺页母模式类型	硬件/应用触发的具体原因	OS 内核接管后的核心处理动作
Demand-zero / 延迟分配	地址合法，但应用首次访问，物理页尚未真正分配	分配空闲 Frame，物理数据清零，更新可用 PTE
File-backed 按需页	代码/数据/mmap 页合法，但磁盘文件内容尚未载入	从文件系统 / 页缓存读取数据，建立页表映射
Swap-in 页面换入	页面曾驻留主存，先前因内存压力被换出到 Swap 区	从 Buddy 分配 Frame，从 Swap 磁盘读回，恢复 PTE
Copy-on-Write (COW)	父子进程共享只读映射，某一进程试图写入该页	检查引用计数，若 > 1 则复制物理 Frame 并置为可写
Protection Fault 权限错	页面映射存在，但本次访问类型 (R/W/X) 违规	校验合法机制 (如 COW 修复)；不可修复则报 SIGSEGV

缺页母模式类型	硬件/应用触发的具体原因	OS 内核接管后的核心处理动作
Invalid Address 越界	虚拟地址根本不属于任何合法 VMA 区域	软件无法修复，内核向进程发送 SIGSEGV 信号终止进程

408 必须形成的 Page Fault 观

“缺页”在经典请求分页题中常特指页面不在主存；但从完整 OS 机制看，page fault 是更广义的异常入口。做 408 题时先服从题目采用的经典模型；做系统理解时再区分 demand paging、COW、permission 等不同 fault 原因。

8 一个 Page 的一生：从不存在到驻留、变脏、被回收

8.1 Page Lifecycle 是虚拟内存最重要的动态模型

一个匿名 heap page 的生命周期可以写成：

```
VMA exists → PTE absent → first touch fault → resident clean
                → dirty / accessed → reclaim candidate → evicted / nonresident
```

未来再次访问时，再经 fault 恢复 residency。

8.2 Resident 与 Valid 是两个容易混淆的词

不同教材/体系结构对 valid/present 命名并不完全一致，因此考试时要看题目定义。方法上应分别记录：

- 地址是否合法属于进程；
- 页面当前是否驻留 RAM；
- 访问权限是否允许。

不要只看到一个“valid bit”就把三层语义压成一层。

9 Demand Paging：为什么“需要时再做”通常更划算

9.1 Lazy 是一种通用系统策略

立即为所有潜在页面分配 RAM 会浪费：

- 从未访问的 heap 页面；
- fork 后马上 exec 的大量页面；
- 稀疏数组的巨大空洞；
- 可执行文件中实际从未执行/读取的区域。

于是系统把成本延迟到第一次真正使用：

Reserve now, pay on first touch.

9.2 代价：首次访问被推入慢路径

Lazy allocation 并没有消灭成本，而是把：

allocation + initialization + mapping

从申请时移动到了首次访问时。因此平均性能是否更好，取决于“有多少申请最终真的会使用”。

10 页面置换：RAM 不够时谁应该留下

10.1 先问为什么需要 replacement

只要 resident working sets 的总需求长期逼近/超过可用 frames，就必须回收一些页面。Replacement 是选择 victim 的 policy，而 page fault / page-in / PTE update 是实现机制。

10.2 四个经典策略的思想，而不是口号

置换算法	核心假设与选择规则	算法关键性质与工程评价
OPT (最佳置换)	淘汰未来最长时间内不会被访问的页面	理想极限下界；需已知未来访存序列，用于评估性能上界
FIFO (先进先出)	淘汰最早进入内存的页面	简单但盲目；可能出现 Belady 异常（框增多缺页反增）
LRU (最近最少使用)	淘汰最长时间未被访问的页面	利用时间局部性；精确维护访问次序的实现成本较高
CLOCK (时钟算法)	用 Reference Bit 近似 LRU (Second Chance)	工程可行性极佳；访问页重置标志，再给一次机会

10.3 Dirty Page 为什么影响置换代价

若 victim 是 clean page，若有可靠后备对象，通常可以直接丢弃映射并未来重新获得；dirty page 则可能需要 write-back：

$$T_{\text{fault}} \approx T_{\text{victim write}} + T_{\text{page in}} + T_{\text{restart}}.$$

所以题目给 dirty bit 时，通常在暗示“换出成本不同”。

10.4 Frame Allocation 与 Replacement Scope

还要区分：

- 每个进程分多少 frames：fixed / variable allocation；
- victim 只能从自己页面选还是可从全局选：local / global replacement。

这是“资源配额”与“局部选择策略”两个层次。

11 Working Set 与 Thrashing: 负载过高时越调度越慢

11.1 局部性为什么支持虚拟内存

程序通常在某一时间窗口只频繁使用较小页面集合，称为工作集的思想基础。只要这些热点页大部分驻留，page fault rate 就可以保持较低。

11.2 Thrashing 的本质

若：

$$\sum_i WorkingSet_i > AvailableFrames,$$

系统会不断：

$$Fault \longrightarrow Evict \longrightarrow Run\ a\ little \longrightarrow Fault \longrightarrow Evict.$$

于是大量时间花在移动页面，而不是有用计算。

跨科统一：Thrashing 与网络拥塞、锁竞争属于同一种系统现象

当并发工作量超过稀缺资源承载能力，增加任务不再增加吞吐，反而提高管理和等待成本。解决思路不是“再多塞几个任务”，而是控制 admission、减少工作集、增加资源或改变调度策略。

12 物理内存管理：Buddy 应该放在这里，而不是和 malloc 混在一起

12.1 历史连续分区与 Free-Space Management

First Fit、Next Fit、Best Fit、Worst Fit、Quick Fit 等算法回答的是：

面对多个空闲连续块，选择哪一个满足连续请求？

它们主要研究查找速度、外部碎片、回收合并等权衡。作为 408 历史模型应掌握，但不要把它们直接等同于现代进程虚拟内存的物理页映射。

12.2 Buddy System: 管理成组连续物理页

Buddy 以 2^k 页为阶数组织空闲块。请求需要 2^k 时：

1. 若该 order 有空闲块，直接分配；
2. 否则从更高 order 分裂；
3. 释放后若 buddy 同阶且空闲，就递归合并。

若基于按 2^k 对齐的地址表示，伙伴定位常可用：

$$buddy_addr = addr \oplus 2^k$$

(具体公式中的单位必须与 $addr$ 和 2^k 的定义一致)。

工程定位

Linux 内存管理把 Page Allocation、Page Tables、Process Addresses、Page Reclaim、Swap、页缓存等作为相互关联但不同的子系统；Buddy 属于物理页分配层，而不是用户态 `malloc()` 的直接算法。

12.3 为什么还需要 Slab/SLUB 一类对象分配器

Buddy 擅长按页/连续页管理物理内存，但内核经常需要大量小型固定结构 (task、inode、dentry 等)。若每个小对象都独占整页会浪费，于是在页分配器之上再建立对象 cache。这再次证明：

物理页分配 $\neq$ 对象分配

13 共享、Copy-on-Write 与 fork：用权限位把复制推迟到真正写入**13.1 fork 的语义承诺**

子进程必须看到“创建时与父进程相同”的初始内存内容，但这并不要求立即复制每个物理页。

13.2 COW 的完整状态转换

Fork 前：

$$P : VPN_1 \rightarrow PFN_8 \quad (RW)$$

Fork 后：

$$P : VPN_1 \rightarrow PFN_8 \quad (RO/COW)$$

$$C : VPN_1 \rightarrow PFN_8 \quad (RO/COW)$$

若 child 首次写：

$$Store \rightarrow Protection\ Fault \rightarrow Kernel\ COW\ Handler$$

然后：

$$Allocate\ PFN_{15} \rightarrow Copy(PFN_8) \rightarrow C : VPN_1 \rightarrow PFN_{15}(RW).$$

父进程仍可继续指向原页。

COW 的真正思想

COW 不是“共享内存 IPC”，而是：先共享物理实现，继续向每个进程承诺私有内存语义；只有写入会破坏私有性时才复制。

13.3 为什么 fork + exec 特别受益

若 child 很快 `exec()`，原地址空间会被新程序映像替换；立即复制父进程全部内存的工作大多白费。COW 将这部分复制完全避免。

14 mmap：虚拟内存与文件系统真正握手的地方

14.1 先分 anonymous 与 file-backed

- Anonymous mapping: heap/stack/匿名 mmap，内容通常不来自普通文件；首次使用可 demand-zero，内存压力下可涉及 swap。
- File-backed mapping: VMA 关联文件和 offset，页面内容由文件作为后备对象。

14.2 mmap 的核心不是“把整个文件读进内存”

更准确的模型是建立：

VirtualPage ↔ FileOffset

关系。首次访问某个尚未驻留的 file-backed page 时：

VA → PageFault → File/PageCache → PhysicalFrame → PTE.

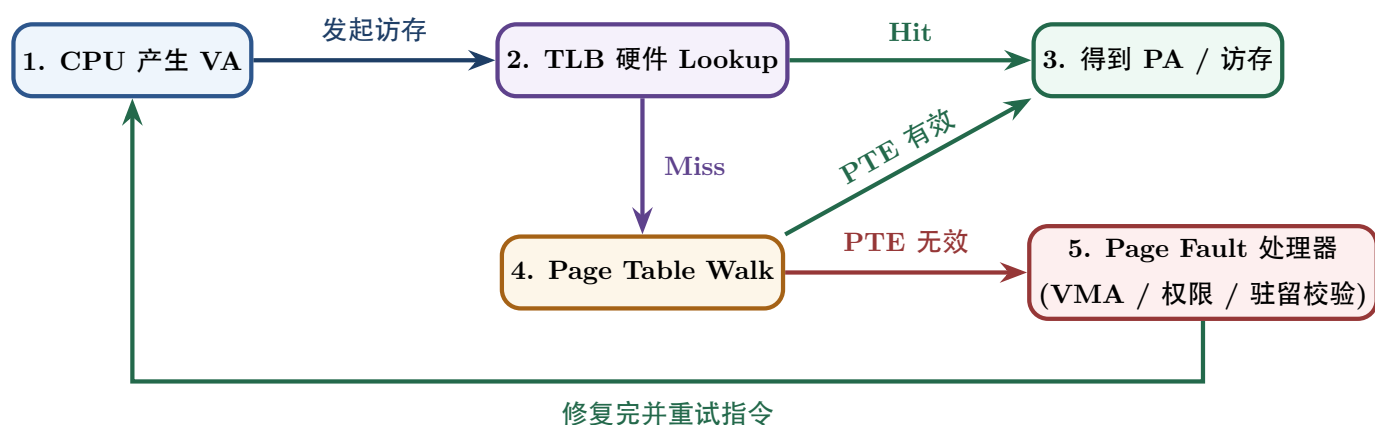
14.3 标准 read 与 mmap 的结构对比

维度 / 特性	标准 read(fd, buf, n) 系统调用	文件映射 file-backed mmap()
应用接口样式	每次数据读取触发显式系统调用	一次映射，之后像常规内存访问一样 load/store
数据内核路径	页缓存 → 显式复制 → 用户缓冲区	文件页对应的物理页可通过页表映射到用户地址空间
数据复制开销	通常需要把页缓存中的数据复制到用户缓冲区	可避免这次显式复制，但缺页、页表维护和一致性仍有成本
控制方式	适合流式读写和显式控制读取范围	适合随机访问、共享映射及按页访问文件内容

不要写“标准 I/O 没有 backing store”

文件内容本身的后备对象一直是文件；区别在于 read() 通常把文件数据复制到用户的匿名 buffer，而 file-backed mmap() 直接把虚拟页与文件内容联系起来。讨论 backing 时必须说明“是哪一个页面/对象的 backing”。

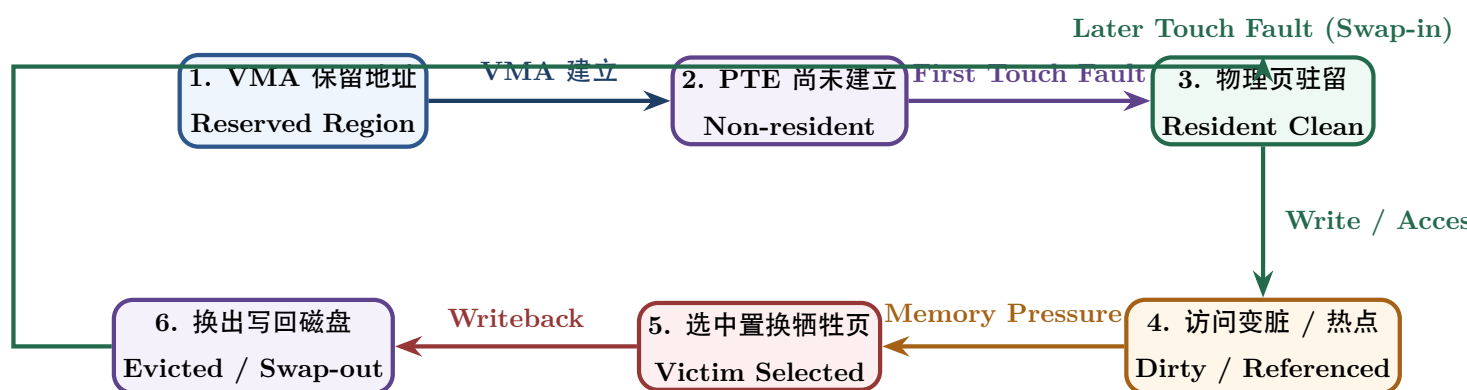
15 综合题一：一次虚拟地址访问



综合题推演时必须记录五类状态

1. 当前地址空间/VMA 是否允许这个 VA；
2. TLB 是否已有 translation；
3. PTE 的 mapping、permission、present 状态；
4. physical frame 是否 resident、clean/dirty、referenced；
5. 若不 resident，backing object 在哪里。

16 综合题二：一个页面的生命周期



这张图把“请求分页、reference/dirty bit、置换、swap/writeback、再次缺页”从多个章节重新还原成一个对象的生命周期。

17 综合题三：fork + COW 的状态变化

手推 COW 固定四步

1. 共享前：谁映射到哪个 PFN?
2. fork 后：父子 PTE 指向同一 frame，但写权限怎样改变？引用计数如何变化？
3. 写 fault：是谁写？MMU 为什么 fault？内核如何判断这是 COW 而不是非法写？
4. 修复后：谁得到新 frame，旧 frame 谁继续引用，权限如何恢复？

18 408 题型地图：每一类题到底在考哪一层

408 常见题型分类	真正考察的系统层次	考场标准推理与定性动作
地址拆分与页号	Paging Math (数学模型)	Page Size → Offset Bits → VPN 位数划分
多级页表空间计算	Sparse Mapping Structure	每级 Index 位数 → PTE 数 → 只为已用区域建低级页表
TLB / EAT 计算	Translation Cache (硬件加速)	画 Hit / Miss 路径，精准数 TLB、页表与主存访问次数
缺页次数模拟	Residency Transitions (驻留)	按访存串动态维护 Resident Set 框状态与置换规则
FIFO/LRU/CLOCK	Replacement Policy (策略)	每次访问后及时更新 Queue 位置、Timestamp 或 Ref Bit
Belady 异常分析	Policy Anomaly (算法异常)	比较框数增加时 FIFO 缺页数反常增加的特例序列
页面分配策略	Resource Allocation (配额)	明确 Fixed / Variable 框架与 Local / Global 置换作用域
COW 机制分析	Permission + Sharing + Fault	校验父子共享只读 PFN、写 Fault、复制 Frame、更新 PTE
连续分区管理	Free-space Management	FF / BF / WF / NF 找块规则，合并空闲区与计算外部碎片
Buddy 系统分配	Physical Page Allocator	计算阶数 $Order\ 2^k$ ，执行分裂 Split 与按位 XOR 伙伴合并
TLB + Cache 综合	OS × 计组软硬件协作	先 VA → PA 翻译，再按 PA 访问 Cache，严格按题目假设推导

19 408 虚拟内存十二问：考场固定协议

拿到任何地址翻译/虚存综合题，依次问

1. 当前讨论的是虚拟空间、页表映射、物理页分配，还是页面置换？
2. VA/PA 各多少位？page size 多少？
3. offset 有几位？VPN/PFN 如何拆？
4. 页表几级？每级 index 如何取得？
5. TLB 是否命中？若 miss，题目规定 page walk 需要几次访问？
6. PTE 是否 present/valid？本次 R/W/X 权限合法吗？
7. 是否 page fault？fault 是“页面不驻留”还是 COW/权限/非法地址？
8. 若需物理 frame，当前有 free frame 吗？
9. 若需 replacement，用什么 policy，victim 作用域是 local 还是 global？
10. victim 是否 dirty，需要额外 write-back 吗？
11. fault 修复后，原指令是否会重试？最终状态怎样更新？
12. 最后再算时间：到底统计了几次 TLB、页表、Cache/Memory、Disk 访问？

20 高频错因诊断：错一道题到底是哪层模型坏了

高频错误表现	模型理解根因漏洞	考场自我纠偏追问
把 malloc() 当物理页申请	概念层级混淆	当前讨论的是 Runtime Chunk、VMA 还是 Physical Frame?
TLB Miss 就判缺页 Page Fault	Fast / 慢路径混淆	PTE 是否其实有效且 Resident? 硬件 Walk 成功则不缺页!
Page Fault 一律去磁盘读盘	Fault 分类缺失	Demand-zero / COW / File / Swap / Invalid 哪一种?
页表项存在就判页面在 RAM	Mapping / Residency 混淆	PTE 的 Present / Valid 标志位状态是什么?
地址拆分计算漏 Offset	翻译对象范围错误	页表只翻译 VPN, Offset 保持不变原样拼接!
LRU / FIFO 模拟算错	状态更新时机错误	每次 Reference 后队列/时间戳何时更新?
Dirty Bit 被忽略	换出成本模型缺失	Victim 是 Clean 还是 Dirty? 是否触发额外 Write-back 延迟?
COW 当共享内存 IPC	语义目标混淆	COW 最终承诺的是私有独立写语义还是共享写?
mmap 当“整文件装入 RAM”	映射与驻留混淆	建立 VA-File 映射关系不等于所有物理页立刻 Resident!
Buddy 与 Best Fit 混淆	Allocator 层级混淆	讨论的是连续进程分区还是物理内存 Order 页框管理?

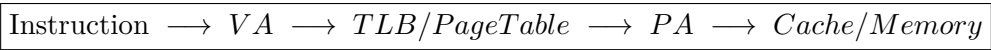
21 教材模型与现代工程：哪些细节要分层记忆

机制主题	408 考研核心教学模型	现代 Linux 内核工程真实实现
页表数据结构	1 / 2 / 多级页表, VPN → PFN 直译	Linux 抽象 5 级页表 (pgd/p4d/pud/pmd/pte), 按架构自动折叠
TLB 一致性	切换地址空间 (切 CR3) 清空 TLB	使用 ASID / PCID 进程上下文标签, 避免全量 Flush 带来的高开销
Page Fault 异常	请求分页缺页、访问越界	广泛依赖 Fault 实现 Lazy Alloc, COW, File-backed mmap, KSM 等
页面置换算法	FIFO / LRU / CLOCK / OPT	采用 Active / Inactive 双链表与 Multi-Gen LRU (MGLRU) 估计热度
物理页管理	Buddy 伙伴系统分配阶数页	在 Buddy 之上建立 SLAB / SLUB 对象缓存器, 专供小内核对象
文件映射机制	mmap + Page Fault 按需加载	与页缓存、VFS 回写机制与 Direct I/O 深度耦合

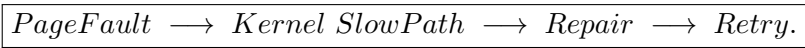
22 跨科桥梁：计组与 OS 在一次访存中怎样接力

软硬件状态所有权		
访存对象 / 动作	CPU / MMU 硬件职责	OS Kernel 软件职责
VA 产生	指令执行单元产生 Effective / Virtual Address	决定进程地址空间布局 (mm_struct) 与允许的 VMA 区域
TLB 缓存	硬件 Lookup 查询并高速缓存 VPN → PFN	在修改页表或销毁进程时负责向 CPU 发送 Flush 命令
Page Table 页表	硬件 MMU 按 ISA 规范自动 Page Walk	物理内存中分配页表页、填写 PTE、配置权限与 Valid 标志
Fault 异常	硬件检测无法完成翻译或权限违规并触发 Exception	捕获异常、分类处理 (Alloc/Disk/COW/Kill)、更新页表与状态
Physical Memory	Cache / Memory Hierarchy 硬件完成物理数据读写	管理全局 Frame 框配额、运行 Page Reclaim 与 Swap 写回

因此完整路径应该始终看成：



若 translation 无法正常完成，才出现：



23 训练阶梯：从会算地址到真正理解虚拟内存

- L1. 能区分 VA、PA、VPN、PFN、offset；
- L2. 能完成单级/多级页表地址拆分；
- L3. 能区分 TLB miss、cache miss、page fault；
- L4. 能手推一次 page fault 的完整状态变化；
- L5. 能模拟 FIFO/LRU/CLOCK 和 frame allocation；
- L6. 能手推一个 page 的 residency 生命周期；
- L7. 能推演 fork+COW、lazy allocation、file-backed mmap；
- L8. 能把“地址空间、页表、物理页、页缓存、文件”放到同一张系统图上，并解释每层状态由谁维护。

24 六条核心结论

复原主干

1. 地址是一种名字，不是位置。虚拟内存首先是把程序命名与物理放置解耦。
2. 页表是一层间接映射。多一次查找，换来隔离、共享、保护、稀疏和延迟分配。
3. Allocation、Mapping、Residency、Translation 是四件不同的事。
4. TLB 是 translation cache；Page Fault 是慢路径 entry。
5. 请求分页靠局部性成立；置换是在 RAM 不足时选择谁继续 resident。
6. COW 与 mmap 都是在重新设计“虚拟页背后对应什么对象”。前者延迟私有复制，后者把文件页直接纳入地址空间。

参考资料与工程边界

- Linux Kernel Documentation, *Memory Management Documentation*: <https://www.kernel.org/doc/html/latest/mm/index.html>
- Linux Kernel Documentation, *Process Addresses*: https://cdn.kernel.org/doc/html/latest/mm/process_addr.html
- Linux Kernel Documentation, *Page Tables*: https://cdn.kernel.org/doc/html/latest/mm/page_tables.html
- MIT PDOS, *xv6: a simple, Unix-like teaching operating system* (2025 edition): <https://pdos.csail.mit.edu/6.828/2025/xv6/book-riscv-rev5.pdf>
- OSTEP / CS537 teaching materials: address space, address translation, paging, TLB and swapping chapters: <https://pages.cs.wisc.edu/~remzi/OSTEP/>

与文件系统的接口

这里负责解释文件怎样成为虚拟页的后备对象。路径名、dentry、inode、打开文件描述、文件偏移、索引分配与磁盘布局由文件系统专题定义；两侧通过 `read()` 与 `mmap()` 的数据路径连接。